Infinite-Horizon Stochastic Optimal Control

Arihant Jain
Electrical and Computer Engineering
UC San Diego

***Abstract*—This project covers solutions to the reference tracking problem for a differential drive car using two different controllers, the certainty equivalent control (CEC) finite-horizon controller and the generalized policy iteration (GPI) infinite horizon controller. We design both controllers to follow a reference trajectory while avoiding obstacles, and attempt to minimize any difference in relative position and orientation.**

I. Introduction

In this paper, we discuss methods to find the optimal (or near-optimal) control input for reference tracking in a system with noise. We have a differential-drive car, which is meant to follow a reference car in a figure-8 trajectory while attempting to avoid obstacles. We are also given a base P controller, which takes the error betweeen the robot state and reference state as the input control.

We then implement two control algorithms, the certainty equivalent control (CEC) finite-horizon controller and the generalized policy iteration (GPI) infinite horizon controller. When using the CEC controller, we ignore noise, and assume a specific fixed-horizon period. The GPI algorithm does not assume a finite horizon, and also takes noise into account. The biggest differentiator between the two algorithms is that CEC computes the optimal control online, while GPI does precomputation offline before querying the stationary optimal policy for values online. Finally, we compare the performance of all 3 controllers using time complexity and solution accuracy.

II. Problem Formulation

The objective is to design a control policy for the differential-drive robot to track a desired reference position trajectory $r_t \in \mathbb{R}^2$ and orientation trajectory $\alpha_t \in [-\pi, \pi)$ while avoiding collisions with obstacles in the environment. There are four circular obstacles, each with radius 0.5: C1 centered at $(2.35, 0.95)$, C2 centered at $(-2.35, -0.95)$, C3 centered at $(1.0, 0)$, and C4 centered at $(-1.0, 0)$. The robot body is modeled as a Euclidean ball $B_{0.3}(p_t) := \{z \in \mathbb{R}^2 | ||z - p_t|| \leq 0.3\}$ centered at p_t with radius 0.3. Let $F := [-3, 3]^2 \backslash U_{q \in (C1 \cup C2 \cup C3 \cup C4)} B_{0.3}(q)$ denote the free space in the environment.

A. Robot dynamics

Consider a robot whose state $x_t = [p_t, \theta_t]^\top$ at discrete-time $t \in \mathbb{N}$ consists of its position $p_t \in \mathbb{R}^2$ and orientation $\theta_t \in [-\pi, \pi)$. The robot is controlled by a velocity input $u_t = [v_t, \omega_t]^\top$ consisting of linear velocity $v_t \in \mathbb{R}$ and angular velocity (yaw rate) $\omega_t \in \mathbb{R}$. The discrete-time kinematic model of the differential-drive robot obtained from exact integration of the continuous-time kinematics with time interval $\Delta > 0$ is:

$$\mathbf{x}_{t+1} = \begin{bmatrix} \mathbf{p}_{t+1} \\ \theta_{t+1} \end{bmatrix} = f(\mathbf{x}_t, \mathbf{u}_t, \mathbf{w}_t) = \begin{bmatrix} \mathbf{p}_t \\ \theta_t \end{bmatrix} +$$

$$\begin{bmatrix} \Delta \operatorname{sinc}\left(\frac{\omega_t \Delta}{2}\right) \cos\left(\theta_t + \frac{\omega_t \Delta}{2}\right) & 0 \\ \Delta \operatorname{sinc}\left(\frac{\omega_t \Delta}{2}\right) \sin\left(\theta_t + \frac{\omega_t \Delta}{2}\right) & 0 \\ 0 & \Delta \end{bmatrix} \begin{bmatrix} v_t \\ \omega_t \end{bmatrix} + \mathbf{w}_t, t = 0, 1, 2, \ldots \tag{1}$$

where $w_t \in \mathbb{R}^3$ models the motion noise with Gaussian distribution $N \quad (0, diag(\sigma)^2)$ with standard deviation $\sigma = [0.04, 0.04, 0.004]^\top \in \mathbb{R}^3$. The motion noise is assumed to be independent across time and of the robot state x_t. The kinematics model in (1) defines the probability density function $pf(x_{t+1}|x_t, u_t)$ of x_{t+1} conditioned on x_t and u_t as the density of a Gaussian distribution with mean $f(x_t, u_t, 0)$ and covariance $diag(\sigma)^2$. The control input ut of the vehicle is limited to an allowable set of linear and angular velocities $U := [0.1, 1] \times [-1, 1]$.

B. Error State

It will be convenient to define an error state $e_t := (\tilde{p}_t, \tilde{\theta}_t)$, where $\tilde{p}_t := p_t - r_t$ and $\tilde{\theta}_t := \theta_t - \alpha_t$ measure the position and orientation deviation from the reference trajectory, respectively. The equations of motion of the error state are:

$$\mathbf{e}_{t+1} = \begin{bmatrix} \tilde{\mathbf{p}}_{t+1} \\ \tilde{\theta}_{t+1} \end{bmatrix} = g(t, \mathbf{e}_t, \mathbf{u}_t, \mathbf{w}_t) =$$

$$\begin{bmatrix} \tilde{\mathbf{p}}_t \\ \tilde{\theta}_t \end{bmatrix} + \begin{bmatrix} \Delta \operatorname{sinc}\left(\frac{\omega_t \Delta}{2}\right) \cos\left(\tilde{\theta}_t + \alpha_t + \frac{\omega_t \Delta}{2}\right) & 0 \\ \Delta \operatorname{sinc}\left(\frac{\omega_t \Delta}{2}\right) \sin\left(\tilde{\theta}_t + \alpha_t + \frac{\omega_t \Delta}{2}\right) & 0 \\ 0 & \Delta \end{bmatrix} \begin{bmatrix} v_t \\ \omega_t \end{bmatrix} +$$

$$\begin{bmatrix} \mathbf{r}_t - \mathbf{r}_{t+1} \\ \alpha_t - \alpha_{t+1} \end{bmatrix} + \mathbf{w}_t, t = 0, 1, 2, \ldots \tag{2}$$

C. Value function

We formulate the trajectory tracking with initial time τ and initial tracking error $\mathbf{e}$ as a discounted infinite-horizon stochastic optimal control problem:

$$V^*(\tau, \mathbf{e}) = \min_{\pi} \mathbb{E}\Bigg[\sum_{t=\tau}^{\infty} \gamma^{t-\tau}(\tilde{\mathbf{p}}_t^\top \mathbf{Q}\tilde{\mathbf{p}}_t + q(1-\cos(\tilde{\theta}_t))^2 + \mathbf{u}_t^\top \mathbf{R}\mathbf{u}_t), \Bigg| , \mathbf{e}_\tau = \mathbf{e}\Bigg]$$
$$\text{s.t.} \quad \mathbf{e}_{t+1} = g(t, \mathbf{e}_t, \mathbf{u}_t, \mathbf{w}_t),$$
$$\mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, , \mathrm{diag}(\boldsymbol{\sigma}^2)), \quad t = \tau, \tau+1, \ldots$$
$$\mathbf{u}_t = \pi(t, \mathbf{e}_t) \in \mathcal{U}$$
$$\tilde{\mathbf{p}}_t + r_t \in \mathcal{F} \tag{3}$$

where $Q \in \mathbb{R}^{2\times 2}$ is a symmetric positive-definite matrix defining the stage cost for deviating from the reference position trajectory r_t, $q > 0$ is a scalar defining the stage cost for deviating from the reference orientation trajectory α_t, and $R \in \mathbb{R}^{2\times 2}$ is a symmetric positive-definite matrix defining the stage cost for using excessive control effort.

III. Methods and Approach

A. CEC

For CEC, we simplify the problem into a finite-horizon problem, and solve it as an optimization problem using casadi.

The modified value function is as follows:

$$V^*(\tau, \mathbf{e}) = \min_{\mathbf{u}_{\tau,\ldots\tau+T-1}} q(\mathbf{x}_{\tau+T}) + \sum_{t=\tau}^{\tau+T-1} \gamma^{t-\tau} \Big(\tilde{\mathbf{p}}_t^\top \mathbf{Q}\tilde{\mathbf{p}}_t + q(1-\cos(\tilde{\theta}_t))^2 + \mathbf{u}_t^\top \mathbf{R}\mathbf{u}_t\Big),$$
$$\text{s.t.} \quad \mathbf{e}_{t+1} = g(t, \mathbf{e}_t, \mathbf{u}_t, 0), \quad t = \tau, \tau+1, \ldots$$
$$\mathbf{u}_t = \pi(t, \mathbf{e}_t) \in \mathcal{U}$$
$$\tilde{\mathbf{p}}_t + \mathbf{r}_t \in \mathcal{F} \tag{3}$$

Using this modified value function, we defined the following NLP:

Optimization Variables:

Control input: $\mathbf{u}_{\tau,\ldots,\tau+T-1}$

Error state: $\mathbf{x}_{\tau+1,\ldots,\tau+T}$

Obstacle avoidance slack: $\mathbf{s}_{\tau+1,\ldots,\tau+T}$

We then subject these variables to 3 constraints:

$$\mathbf{x}_{t+1} = g(t, \mathbf{x}_t, \mathbf{u}_t, 0) \quad \forall t = \tau, \tau+1, \ldots, \tau+T-1$$

$$\mathbf{u}_t \in U \quad \forall t = \tau, \tau+1, \ldots, \tau+T-1$$

$$||\mathbf{x}_t - obs|| < (0.5 + 0.3 - \mathbf{s}_t) \quad \forall t = \tau+1, \tau+2, \ldots, \tau+T$$

The objective function can then minimize our modified value function, with an additional objective to minimize the slack:

$$\text{objective} := V^*(\tau, \mathbf{e}) + \min(\sum_{\tau+1}^{\tau+T} s_t)$$

B. GPI

For GPI, we need to first discretize our state space to be able to efficiently compute solutions. We discretize the state and control spaces into $(nt, nx, ny, n\theta) = (100, 13, 13, 19)$ and $(nv, n\omega) = (10, 10)$ number of grid points, respectively. We select this level of discretization due to memory bandwidth and compute limitations. Our space is not linearly discretized since we want to allow higher resolution around lower errors, and lower resolution around higher error. We do this to ensure that cells actually change with a change in the error, which wouldnt take place in a linear coarse grid.

$ex_space = [-6.0, -2.0, -1.0, -0.50, -0.25, -0.10, 0.0, 0.10, 0.25, 0.50, 1.0, 2.0, 6.0]$

$ey_space = [-6.0, -2.0, -1.0, -0.50, -0.25, -0.10, 0.0, 0.10, 0.25, 0.50, 1.0, 2.0, 6.0]$

$eth_space = \pi*[-1.00, -0.75, -0.50, -0.35, -0.25, -0.18, -0.12, -0.08, -0.04, -0.02, 0.00, 0.02, 0.04, 0.08, 0.12, 0.18, 0.25, 0.35, 0.50, 0.75, 1.00]$

In this problem, we use the infinite-horizon optimal value function, over which we add a loss term for obstacle collision, which heavily penalizes collisions if they take place:

$$10^6 * (||\mathbf{e}_t + \mathbf{ref}_t - obs|| < (0.5 + 0.3))$$

To account for the noise, the transition matrix contains not only the indexes for the no-noise state transition, but also the 26 adjacent state indices and probabilities associated with them. Our state space is scaled for an error between -3 and 3 metres, and the full range of angular error. Since in one timestep of 0.5sec the robot can only move a maximum of 0.5m, we should keep our discretization to consider a delta of 0.5, which is how we chose our ranges and discretization points.

We also chose 20 policy evaluation iterations per policy improvement iteration, and a total of 50 policy improvement iterations. This was sufficient to enable accurate control outputs. (unfortunately not true):)

The time complexity of precomputation was $O(T \times n_x \times n_y \times n_{theta} \times n_v \times n_w \times 27)$. Similarly, for policy evaluation and policy improvement, we iterated over all values in $\mathbf{V}$ and $\pi_{\mathbf{t}}$, which lead to a complexity of $O(T \times n_x \times n_y \times n_{theta} \times n_v \times n_w)$ for policy improvement, and $O(T \times n_x \times n_y \times n_{theta})$ for policy evaluation.

IV. Results

Show qualitative and quantitative results for both the CEC and GPI trajectory tracking techniques and compare their performance in terms of computational complexity, collision avoidance, and tracking error. You may discuss the effect of ignoring the noise in the CEC formulation and of the discretization in the GPI formulation. Discuss also the performance and any interesting details you observe about the two techniques

Using images and the below table, we illustrate visual comparisons, computational complexity (in terms of time taken),

and error of each of the three algorithms. The biggest differentiator between the two algorithms is that CEC computes the optimal control online, while GPI does precomputation offline before querying the stationary optimal policy for values online.

Controller	Time Taken (s)	Offline T(s)	Error_trans	Error_rot
P controller	0.03	0	455.94	25.78
CEC	9.67	0	51	40.81
GPI	927	902	358	91.75

TABLE I
COMPARISON OF CONTROLLERS

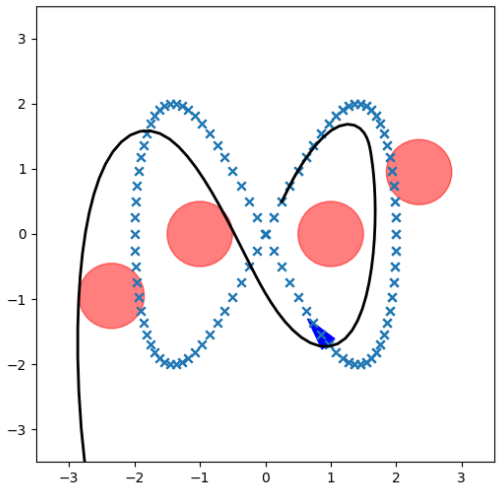
Although these values are from individual experiments, multiple experiments will show similar performance in terms of time taken. However, the error varies significantly with the P controller and GPI due to reasons discussed below.

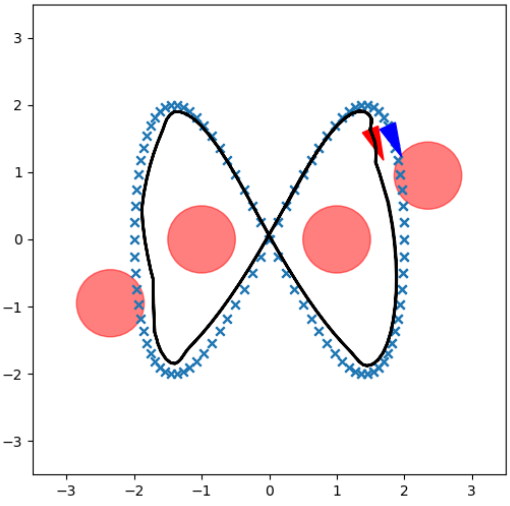
A. Simple P controller

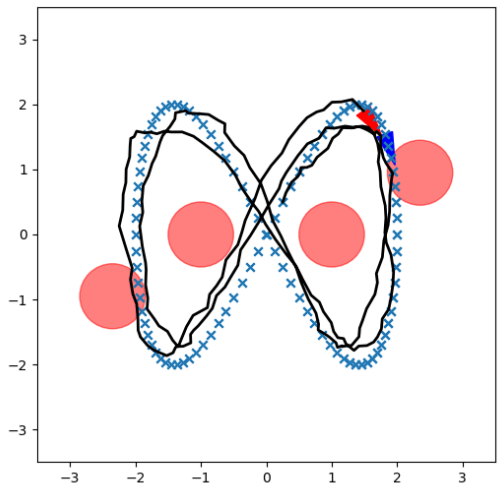
This controller ignores obstacles and returns the valid control input closest to the scaled difference between the current state and reference state. Thus, the translational error is very bad, and the rotational error remains small.

The time taken is minimal since the only computation is a difference between the reference and current state.

An interesting thing is how sensitive the P controller is to the noise. Depending on the noise, the controller might closely follow the reference, or blow up to being really far away seemingly randomly. We can see that from the results attached below:







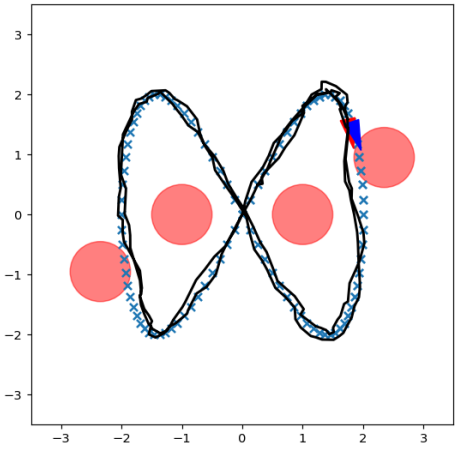
B. CEC

Since we ignore the noise in CEC, we sometimes end up close to/ collide with one of the obstacles due to the noise in the system. This was optimized by increasing the cost of collision, but not fully removed. Even though we ignore noise here, the policy will be close to optimal due to the Gaussian nature of the noise with very low variance.

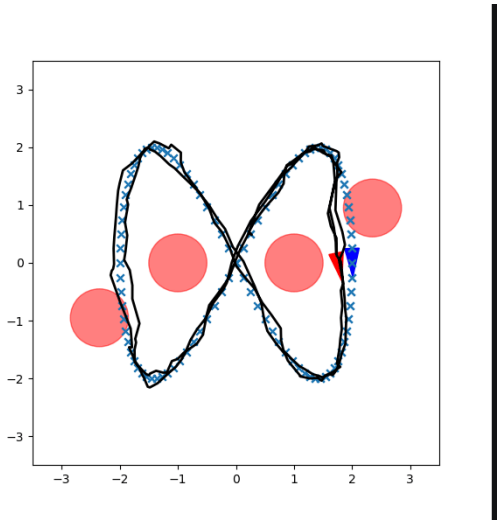
We ended up using the following values for computing the value function:

$\mathbf{Q} = 5 * \mathbf{I}_{2\times 2}$
$\mathbf{R} = 0.001 * \mathbf{I}_{2\times 2}$
$\mathbf{q} = 75$
$\gamma = 0.5$
$\mathbf{T} = 10$

Also, since this algorithm computed the control inputs by solving the NLP online at every iteration, each individual iteration was slow. However, the overall code ran much faster than GPI (while still being slower than the simple P controller).



Here is an example where the CEC algorithm resulted in a crash with the obstacles since noise was not considered in the motion:



C. GPI

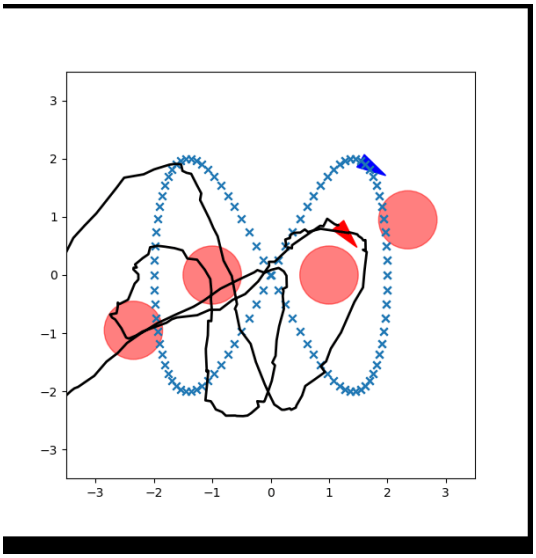
GPI, although theoretically optimal, did not return great results in my implementation. This could be due to errors such as inappropriate state space discretization, minor mistakes in transition matrix precomputation, or some other parameter tuning that I missed. I attempted many optimizations, both to improve time taken offline as well as accuracy, but was unable to stop the jittery motion and incorrect turning.

However, in terms of performance online, the GPI algorithm returned control inputs at almost the same speed as the simple P controller, with the entire bulk of computation taking place offline, before the robot even starts moving. By vectorizing over numpy matrices, we were able to reduce the time taken from over 8000s to just over 900s. This enabled fast computation online, but still did not result in great performance unfortunately.

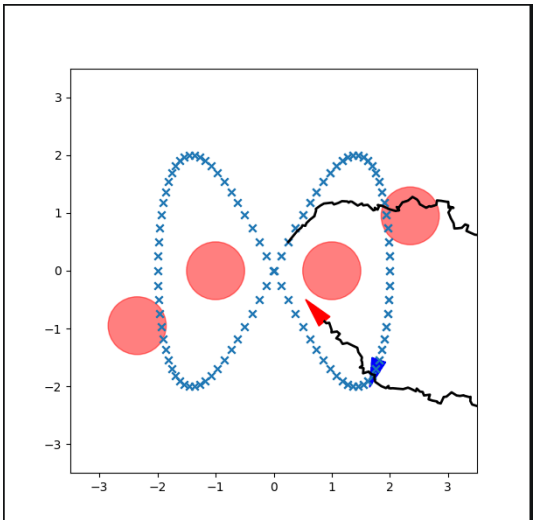
I think the most interesting thing I observed about GPI is the fact that it is incredibly sensitive to the values of $\mathbf{Q}$, $\mathbf{R}$, $\mathbf{q}$, and γ. This, along with the time taken to compute the transition matrices and perform policy iteration steps, led to a large amount of time being spent on tuning rather than debugging errors in the code. The biggest parameter to tune, however, has to have been the discrete spaces, since those determined how well the policy actually performed.

We ended up using the following values for computing the value function:

$\mathbf{Q} = 500 * \mathbf{I}_{2\times 2}$
$\mathbf{R} = 0.0001 * \mathbf{I}_{2\times 2}$
$\mathbf{q} = 100$
$\gamma = 0.8$
$num_evals = 20$
$num_iters = 50$



Below is an example during testing, when the GPI algorithm did not work even remotely closely to the reference trajectory, accumulating an error_trans of 901.98 and an error_rot of 365.68:



V. Conclusion

In this report, we illustrate two unique ways to solve a reference tracking problem, using CEC without considering noise and GPI with noise in consideration. We implement both methods in Python, using casadi for CEC and numpy for GPI, and tune parameters to obtain the best results.

We compare performance in terms of computational complexity, time to output a control value online, and the cumulative positional and rotational error. We also discuss weaknesses of each algorithm, specifically regarding computational complexity, and illustrate results visually for each algorithm. Future work will involve optimizations to the code, specifically regarding GPI, to ensure robust and accurate control.

References

1) Used Prof Atanasov's class notes and links provided in the project description.
2) Spoke about high level concepts with Thomas, specifically mentioning adding slack to the obstacle collision for part 1.